FORWARD DEPLOYED ENGINEER TAKE-HOME

The Lenny Growth Assistant

Product Requirements Document

STATUS	**Draft — Discovery**
DATE	25 August 2026
PURPOSE	Define a grounded conversational assistant, content skill, and secure artifact-generation experience built on Lenny's Podcast transcripts.

Working product specification • Version 0.1

This PRD captures the product requirements, working assumptions, explicit trade-offs, and unresolved decisions for an AI-powered internal assistant based on Lenny's Podcast transcripts. The target outcome is a system that another team can understand, run, test, and extend.

Document field	Value
Status	Draft — Discovery
Assignment	Forward Deployed Engineer Take-Home
Last updated	25 August 2026
Primary audience	Product, growth, and engineering evaluators
Decision state	Core requirements defined; architecture choices in progress

Contents

1 Product Overview

The Lenny Growth Assistant is an AI-powered internal assistant that enables product and growth teams to interact with knowledge drawn from Lenny's Podcast transcripts.

The product will:

- Let users ask product and growth questions conversationally through a chat interface.
- Return answers grounded in the available transcript corpus using retrieval-augmented generation (RAG).
- Continue conversations while preserving independent session context in a database.
- Transform transcript-grounded knowledge and conversation context into structured written content, including Ship 30 for 30–style essays.
- Generate Markdown and HTML/CSS artifacts, from concise summaries to small visual or UI pieces.
- Treat generated HTML as untrusted content and isolate or sanitize it before rendering.
- Display generated artifacts directly inside the application.

The product should make reasonable assumptions visible, communicate trade-offs, and leave behind a system another team can operate and extend.

2 Discovery Brief

2.1 Primary User

Status: Completed

The initial audience spans product managers, growth professionals, startup founders, research-oriented product/growth teams, and internal teams looking for actionable product advice.

Builder decision

Design for a broad product audience—from interns and evaluators to beta users and product leads—who want conversational access to Lenny's Podcast knowledge and interactive generated visuals. The experience should emphasize easy setup, straightforward use, safety guardrails, content quality, and application stability.

2.2 User Problem

Status: Completed

Users need grounded answers, reusable written content, and rendered artifacts without having to understand prompting, model selection, or infrastructure.

The product should make it easy to explore Lenny's Podcast knowledge conversationally and convert that knowledge into useful, structured outputs.

2.3 User Jobs-to-be-Done

Status: Completed

Six core scenarios define the initial jobs-to-be-done.

2.3.1 Scenario 1 — Solve a product or growth problem

When I am trying to solve a product or growth problem, I want to find relevant insights from Lenny's Podcast so that I can make a better-informed decision.

2.3.2 Scenario 2 — Discover relevant episodes

When I want to learn about a product or growth topic, I want to discover the most relevant episodes and experts so that I do not have to search through the entire podcast myself.

2.3.3 Scenario 3 — Compare perspectives

When I want to understand a product or growth topic from multiple perspectives, I want to compare insights across Lenny's guests so that I can identify patterns, disagreements, and useful approaches.

2.3.4 Scenario 4 — Explore an expert

When I want to understand an expert's thinking on a topic, I want to explore their relevant podcast conversations interactively so that I can go deeper without manually searching the transcript.

2.3.5 Scenario 5 — Create structured written content

When I have gathered useful product or growth insights, I want to turn them into structured written content so that I can reuse those insights without starting from scratch.

2.3.6 Scenario 6 — Create a usable artifact

When I have useful information from the conversation, I want to turn it into a usable document or artifact so that I can take the knowledge beyond the chat.

3 Product Goals

The product must:

1. Provide reliable answers grounded in Lenny's transcript knowledge.
2. Maintain independent conversational sessions.
3. Allow users to generate useful written content from grounded knowledge.
4. Allow users to generate and view artifacts within the application.
5. Make model and provider configuration understandable to the evaluator.
6. Fail clearly when the available knowledge does not support an answer.
7. Be reproducible and operable by another engineer.

Builder decision

Use Docker for single-command setup. Ground generated content with RAG, use structured outputs where appropriate, and validate generated HTML against the user's request and safety rules. If validation fails, discard the artifact and present a safe fallback. Users should be able to write natural-language requests while the system handles prompt construction, retrieval, reliability checks, and content formatting.

4 Non-Goals

Status: TBD

The final non-goal boundary remains open, but the following areas are excluded from the initial scope.

- General-purpose web search.
- Knowledge outside the supplied transcript corpus.
- Autonomous execution of external actions; this is not a general-purpose autonomous agent.
- Multi-user collaboration or authentication in the initial version.
- Production-scale traffic and production-hardening beyond the take-home requirements.
- Complex document editing; the product is not intended to be a full document editor.
- Persistent user profiles beyond assignment requirements.

5 Core User Tasks

The initial product comprises three major capabilities.

5.1 Grounded Conversational Assistant

Users should be able to:

1. Start a new chat.
2. Ask a product or growth question.
3. Receive an answer grounded in the transcript knowledge base.
4. Ask follow-up questions.
5. Receive answers that preserve session context.
6. See the relevant transcript or source used.
7. Be told when the available material does not support an answer.
8. Generate a reusable structured document, such as an article, information table, or key-point summary.

9. Generate HTML pages, infographics, or Markdown when a visual or structured format communicates the answer more effectively.

The implementation must use a RAG or long-context approach and provide source grounding.

5.2 Ship 30 for 30 Content Skill

Users should be able to request a Ship 30 for 30–style essay based on grounded knowledge. The writing principles will be encoded from *How to Start Writing Online — The Ship 30 for 30 Ultimate Guide*.

The generated content should approximately:

- Contain 1,250 words.
- Begin with a strong hook.
- Maintain clear narrative progression.
- Be easy to skim.
- Use headings, bullets, and selective emphasis.
- Provide a specific, useful takeaway.
- Ground its claims in the transcript knowledge base.

These principles must be implemented as a dedicated skill or tool rather than relying solely on an unstructured prompt.

5.3 Artifact Generation

Users should be able to request Markdown documents and HTML/CSS artifacts, including:

- Small UI pieces.
- Information summaries.
- Key summary points in Markdown.
- Visual representations of information.
- Infographics or visual explanations where appropriate.

Generated artifacts should appear in an Artifact Viewer beside the chat rather than only as raw code or in another application. Generated HTML must be treated as untrusted and rendered through an explicit isolation or sanitization strategy.

6 User Flows

Flow A — Ask a question
New Chat → Ask Question → Retrieve Knowledge → Generate Answer → Show Sources

Flow B — Follow-up
Existing Session → Follow-up Question → Session Context + Relevant Knowledge → Answer

Flow C — Generate essay
Conversation → Request Ship 30 Essay → Grounded Content Generation → Display Result

Flow D — Generate artifact
Conversation → Request Artifact → Generate Markdown/HTML/CSS → Artifact Viewer

Flow E — Unsupported question
Question → Insufficient Knowledge → Explain Limitation → Avoid Unsupported Claim

7 Success Metrics

Status: Completed

The product will track quality, reliability, and operational readiness rather than relying on a single aggregate metric.

7.1 Product Quality

- Percentage of evaluated answers that are correctly grounded.
- Percentage of answers containing an appropriate source citation.
- Percentage of unsupported questions correctly identified rather than hallucinated.

7.2 Reliability

- Successful request completion rate.
- Retrieval failure rate.
- Model failure rate.
- Database failure recovery rate.

7.3 Operational Readiness

- Time required for a fresh evaluator to get the application running.
- Percentage of critical workflows covered by automated tests.

8 Assumptions

Status: TBD

The assignment is intentionally incomplete. Every material implementation assumption will be documented rather than hidden in code.

Assumptions must be made explicit for:

- Primary user and expected usage.
- Transcript corpus and knowledge freshness.
- Authentication and user identity.
- Session lifetime.
- Model availability and local machine capabilities.
- Supported artifact types.
- Security boundaries.
- Expected evaluation environment.

9 Scope

9.1 In Scope

- Conversational assistant with session isolation and context.
- Transcript knowledge base, grounded answers, and source identification.
- Ship 30 for 30 content skill.
- Markdown and HTML/CSS generation with an in-app Artifact Viewer.
- Cloud model and local Ollama configuration.
- PostgreSQL persistence.
- API validation and structured errors.
- Health endpoints, logging, observability, and failure handling.
- Reproducible startup, automated tests, and documentation.

9.1.1 Finalized Technology Decisions

Area	Decision
Backend API	**FastAPI**
Database	**PostgreSQL**
Cloud LLM	**Gemini**
Local LLM	**Ollama**
Local model	**TBD** — select based on machine capabilities and output quality
Agent framework	**TBD**
Frontend	**TBD**
Retrieval implementation	**TBD**
Embedding model	**TBD**
Vector storage/indexing	**TBD**
Deployment topology	**TBD**
Observability stack	**TBD**
Testing stack	**TBD**
CI/CD	**TBD**

9.2 Out of Scope

TBD — to be finalized jointly. The working boundary is defined in Section 4.

10 Risks and Trade-offs

Risk	Impact	Mitigation	State
Hallucinated answers	High	Grounding, source attribution, and refusal behavior	**TBD**
Weak local model	High	Evaluate suitable Ollama model and configuration	**TBD**
Retrieval misses relevant context	High	Retrieval evaluation and tuning	**TBD**
High latency	Med.–high	Measure retrieval and model stages independently	**TBD**
Cloud model cost	Medium	Explicit provider and model configuration	**TBD**
Unsafe HTML	High	Isolation and sanitization strategy	**TBD**
Database failure	High	Graceful failure behavior and health checks	**TBD**
Missing model/API credentials	Medium	Clear configuration and fallback behavior	**TBD**

11 Acceptance Criteria

Status: TBD — finalize after product and architecture decisions

The following checklist defines the current minimum acceptance boundary.

11.1 Conversational Assistant

- ❑ User can start a new session.
- ❑ Sessions maintain independent context.
- ❑ User can ask product and growth questions.
- ❑ Answers are grounded in transcript knowledge.
- ❑ Relevant sources are identified.
- ❑ Unsupported questions are acknowledged appropriately.
- ❑ User can request structured outputs from the conversation.

11.2 Content Skill

- ❑ User can request a Ship 30 for 30–style essay.
- ❑ Output follows the defined content requirements.
- ❑ Claims remain grounded in available knowledge.
- ❑ Ship 30 for 30 writing principles are implemented as a dedicated skill or tool.

11.3 Artifacts

- ❑ User can request Markdown.
- ❑ User can request HTML/CSS.
- ❑ Generated artifacts render inside the application.
- ❑ HTML rendering does not expose the host application to untrusted content.
- ❑ Invalid, unsafe, or broken generated HTML is handled through the defined fallback behavior.

11.4 Models

- ❑ Gemini cloud model can be selected and configured.
- ❑ Ollama/local model can be used for the demo.
- ❑ Provider selection is visible and documented.
- ❑ Failure behavior is understandable.

11.5 Database

- ❑ PostgreSQL stores sessions.

- ❑ PostgreSQL stores conversation messages.
- ❑ Sessions maintain independent context.
- ❑ Database failures are handled gracefully.

11.6 Operations

- ❑ Application can be started through the documented setup process.
- ❑ No secrets are committed.
- ❑ Health endpoints exist.
- ❑ Important failures are observable.
- ❑ Critical workflows have automated tests.

12 Implementation Plan

Technology and architecture decisions will be made progressively after the product requirements and scope are agreed upon. Remaining decision areas include:

- Frontend technology and agent framework.
- Local Ollama model.
- Retrieval architecture, embedding model, and vector storage/indexing.
- Artifact isolation strategy.
- Deployment topology and observability stack.
- Testing strategy and CI/CD.

Requirement → Options → Trade-offs → Decision → Reason

13 Open Decisions

1. **Who exactly is the primary user?** A learner exploring Lenny's Podcast, an intern or evaluator testing the platform, a product lead assessing the UX, or a founder prioritizing clear metrics and stable performance?
2. **What is the user's most important job?** Obtain clear, reliable results and turn them into structured or visual outputs.
3. **What does an ideal interaction look like?** Ask a question, receive a grounded answer, handle irrelevant requests safely, and generate reusable documents or rendered artifacts.
4. **What should the assistant refuse to do?** Answer irrelevant or unsupported queries, or render

broken or unsafe HTML; it should use a safe fallback instead.

5. **What is MVP versus nice-to-have?** The working MVP includes grounded content generation, structured output, conversational memory, and rendered HTML/Markdown. Nice-to-have scope remains open.
6. **What metric proves success?** Select primary thresholds from the quality, reliability, and operational metrics in Section 7.
7. **What assumptions are acceptable?** The initial version is not production-scale; API keys for additional cloud providers will use a bring-your-own-key model.
8. **What failure behavior should users see?** A fallback UI and message tailored to the failure type.

14 Decision Log

Decision	Options considered	Current decision	Reason
Primary user	Broad product audience	**TBD**	Final target persona requires prioritization
Core user job	Grounded answers, structured content, artifacts	**TBD**	Requires product prioritization
MVP scope	Required capabilities versus enhancements	**TBD**	Final boundary not yet agreed
Success metrics	Quality, reliability, and operational metrics	All listed metrics	Provides visibility into product quality and system reliability
Backend API	FastAPI / alternatives	**FastAPI**	Required by assignment and finalized
Database	PostgreSQL / SQLite	**PostgreSQL**	Required by assignment and finalized
Cloud LLM	Gemini / other cloud providers	**Gemini**	Finalized; available through the builder's Google ecosystem
Local LLM	Ollama / other local runtimes	**Ollama**	Mandatory for the assignment demo and finalized
Local model	Candidate Ollama models	**TBD**	Must run comfortably on the development machine
Frontend	Candidate web frameworks	**TBD**	Evaluation pending
Agent layer	Anthropic Claude Agent SDK / Pi Coding Agent	**TBD**	Assignment requires an agent layer; implementation choice remains open
Retrieval	Candidate RAG architectures	**TBD**	Evaluation pending
Embeddings	Candidate embedding models	**TBD**	Evaluation pending
Vector storage	Candidate indexes/stores	**TBD**	Evaluation pending
Artifact isolation	Sandboxing / sanitization / both	**TBD**	Security design pending
Deployment	Candidate topologies	**TBD**	Evaluation pending
Observability	Candidate stacks	**TBD**	Evaluation pending
Testing	Candidate stacks	**TBD**	Evaluation pending
CI/CD	Candidate workflows	**TBD**	Evaluation pending

End of working product requirements document